



数据结构与算法分析



人工智能与自动化学院

韩守东

shoudonghan@hust.edu.cn

13720301586



10. 递归与分治算法

- ▶ 递归概念回顾
- ▶ 分治法基本思想
- ▶ 根据实例掌握分治法
 - ▶ 二分搜索
 - ▶ 大整数的乘法
 - ▶ 矩阵乘法
 - ▶ 合并排序
 - ▶ 棋盘覆盖
 - ▶ 最近点对问题
 - ▶ 循环赛日程表



10.1 递归-概念

- ▶ **递归算法**: 直接或间接地调用自身的算法
- ▶ **递归函数**: 用函数自身给出定义的函数
- ▶ **分治法**产生的子问题往往是原问题的较小模式, 这为使用递归技术提供了方便
 - 反复应用分治手段, 可以使子问题与原问题**类型一致**而其规模却不断缩小, 最终使子问题缩小到很容易直接求出其解
 - 自然导致递归过程的产生
- ▶ **分治与递归**像一对孪生兄弟, 经常同时应用在算法设计之中, 并由此产生许多高效算法



10.1 递归-示例

▶ 例 1: Fibonacci 数列

- 无穷数列 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ……
，称为 Fibonacci 数列。它可以递归地定义为：

$$F(n) = \begin{cases} 1 & n = 0 \\ 1 & n = 1 \\ F(n-1) + F(n-2) & n > 1 \end{cases}$$

边界条件

递归方程

第 n 个 Fibonacci 数可递归地计算如下：

```
1. int fibonacci(int n)
2. {
3.     if (n <= 1) return 1;
4.     return fibonacci(n-1)+fibonacci(n-2);
5. }
```

非递归调用

$$F(n) = \frac{1}{\sqrt{5}} \left[\left(\frac{1+\sqrt{5}}{2} \right)^{n+1} - \left(\frac{1-\sqrt{5}}{2} \right)^{n+1} \right]$$



10.1 递归-示例

► 例2: Ackerman函数

- 当一个函数及它的一个变量是由函数自身定义时，称其为双递归函数

□ Ackerman函数 $A(n, m)$ 定义如下：

$$A(n, m) = \begin{cases} 2 & n = 1, m = 0 \\ 1 & n = 0, m \geq 0 \\ n + 2 & n \geq 2, m = 0 \\ A(A(n - 1, m), m - 1) & n, m \geq 1 \end{cases}$$

例1可以找到非递归定义实现，Ackerman函数则无此类定义



10.1 递归-示例

► $A(n, m)$ 自变量 m 的每一个值都定义了一个单变量函数

- $m=0$ 时, $A(n, 0)=n+2$

- $m=1$ 时, $A(n, 1)=A(A(n-1, 1), 0)=A(n-1, 1)+2$, $A(1, 1)=2$

故根据迭代式: $A(n, 1)=2*n$

- $m=2$ 时, $A(n, 2)=A(A(n-1, 2), 1)=2A(n-1, 2)$, $A(1, 2)=A(A(0, 2), 1)=A(1, 1)=2$

故根据迭代式: $A(n, 2)=2^n$

- $m=3$ 时, 类似的可以推出 $A(n, 3) = 2^{2^{n-2}}$

- $m=4$ 时, $A(n, 4)$ 的增长速度非常快, 以至于没有适当的数学公式来表

示这一函数

$$A(n, m) = \begin{cases} 2 & n = 1, m = 0 \\ 1 & n = 0, m \geq 0 \\ n + 2 & n \geq 2, m = 0 \\ A(A(n-1, m), m-1) & n, m \geq 1 \end{cases}$$

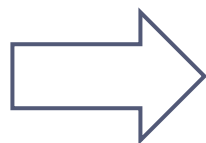


10.1 递归-示例

► 例3：排列问题

设计一个递归算法生成 n 个元素 $\{r_1, r_2, \dots, r_n\}$ 的全排列, 共 $n!$ 种

1, 2, 3



1 2 3, 1 3 2, 2 1 3, 2 3 1, 3 1 2, 3 2 1

6种情况

设 $R=\{r_1, r_2, \dots, r_n\}$ 是要进行排列的 n 个元素, $R_i=R-\{r_i\}$ 。集合 X 中元素的全排列记为 $\text{perm}(X)$ 。 $(r_i)\text{perm}(X)$ 表示在全排列 $\text{perm}(X)$ 的每一个排列前加上前缀得到的排列。 R 的全排列可归纳定义如下:

- 当 $n=1$ 时, $\text{perm}(R)=(r)$, 其中 r 是集合 R 中唯一的元素;
- 当 $n>1$ 时, $\text{perm}(R)$ 由 $(r_1)\text{perm}(R_1)$, $(r_2)\text{perm}(R_2)$, ..., $(r_n)\text{perm}(R_n)$ 构成。



10.1 递归-小结

► 优点

结构清晰，可读性强，而且容易用数学归纳法来证明算法的正确性，因此它为设计算法、调试程序带来很大方便。

► 缺点

递归算法的运行效率较低，无论是耗费的计算时间还是占用的存储空间都比非递归算法要多。



10.1 递归-小结

► 解决办法：在递归算法中消除递归调用，使其转化为非递归算法

● 采用一个用户定义的栈来模拟系统的递归调用工作栈

该方法通用性强，但本质上还是递归，只不过人工做了本来由编译器做的事情，优化效果不明显

● 用递推来实现递归函数

后一种方法在时空复杂度上有较大改善，但适用范围有限



10.2 分治法-基本思想

将一个难以直接解决的大问题，分割成一些规模较小的相同问题，以便各个击破，分而治之。

- ▶ 将待求解的问题分解成规模更小的子问题
 - 若子问题的规模仍然不够小，则再划分为 k 个子问题
 - 以上过程递归进行
 - 直到问题规模足够小，容易求出其解
- ▶ 将小规模问题的解合并为一个更大规模的问题的解
 - 自底向上逐步求出原来问题的解



10.2 分治法-计算步骤

divide-and-conquer (P)

```
{  
  if ( | P | ≤ n0) return adhoc(P); //解决小规模的问题  
  divide P into smaller subinstances P1,P2,...,Pk;  
  //分解问题  
  for (i=1,i≤k,i++)  
    yi=divide-and-conquer(Pi); //递归的解各子问题  
  return merge(y1,...,yk); //将各子问题的解合并为原问题的解  
}
```

平衡(balancing)子问题思想

人们从大量实践中发现，在用分治法设计算法时，最好使子问题的规模大致相同。即将一个问题分成大小相等的k个子问题的处理方法是行之有效的，它几乎总是比子问题规模不等的做法要好。





10.2 分治法-复杂性分析

- ▶ 规模为 n 的问题被分成 k 个规模为 n/m 的子问题：
 - 设分解阈值 $n_0=1$, adhoc解规模为1的问题耗费1个单位时间
 - 设分解为 k 个子问题及用merge将子问题的解合并为原问题需用 $f(n)$ 个单位时间
 - $T(n)$ 表示分治法解规模 $|p|=n$ 的问题所需计算时间

$$T(n) = \begin{cases} O(1) & n = 1 \\ kT(n/m) + f(n) & n > 1 \end{cases}$$

反复迭代



$$T(n) = n^{\log_m k} + \sum_{j=0}^{\log_m n - 1} k^j f(n/m^j)$$



10.2 分治法-适用条件

- ▶ 分治法所能解决的问题一般具有以下几个特征：
 - 规模缩小到一定的程度，可容易解决
 - 子问题为相同问题，具有最优子结构性质
 - 子问题的解可以合并为该问题的解
 - 子问题间相互独立，不含公共子问题

这条特征涉及到分治法的效率，如果各子问题是不独立的，则分治法要做许多不必要的工作，重复地解公共的子问题。



10.3 例1-二分搜索技术

- ▶ 给定已按升序排好序的 n 个元素 $a[0:n-1]$ ，现要在这 n 个元素中找出一特定元素 x 。
- ▶ 分析：
 - 该问题的规模缩小到一定的程度就可以容易地解决；
 - 该问题可以分解为若干个规模较小的相同问题；
 - 分解出的子问题的解可以合并为原问题的解；
 - 分解出的各个子问题是相互独立的。

分析：很显然此问题分解出的子问题相互独立，即在 $a[i]$ 的前面或后面查找 x 是独立的子问题，因此满足分治法的第四个适用条件。



10.3 例1-二分搜索技术

- ▶ 给定已按升序排好序的 n 个元素 $a[0:n-1]$ ，现要在这 n 个元素中找出一特定元素 x 。
- ▶ 据此容易设计出二分搜索算法： $l:left, r:right$;

```
template<class Type>
int BinarySearch(Type a[], const Type& x,
int l, int r)
{
    while (r >= l){
        int m = (l+r)/2;
        if (x == a[m]) return m;
        if (x < a[m]) r = m-1;
        else l = m+1;
    }
    return -1;
}
```

算法复杂度分析：

每执行一次算法的while循环，待搜索数组的大小减少一半。因此，在最坏情况下，while循环被执行了 $O(\log n)$ 次。循环体内运算需要 $O(1)$ 时间，因此整个算法在最坏情况下的计算时间复杂性为 $O(\log n)$ 。



10.3 例2-大整数的乘法

- ▶ 请设计一个有效的算法，可以进行两个n位二进制大整数的乘法运算

▶ 复杂度分析

$$T(n) = \begin{cases} O(1) & n = 1 \\ 4T(n/2) + O(n) & n > 1 \end{cases}$$

$$T(n) = O(n^2)$$

✖ 没有改进

Y =

c

d

$$X = a 2^{n/2} + b$$

$$Y = c 2^{n/2} + d$$

$$XY = ac 2^n + (ad+bc) 2^{n/2} + bd$$



10.3 例2-大整数的乘法

- ▶ 请设计一个有效的算法，可以进行两个n位二进制大整数的乘法运算

▶ 复杂度分析

$$T(n) = \begin{cases} O(1) & n = 1 \\ 3T(n/2) + O(n) & n > 1 \end{cases}$$

$$T(n) = O(n^{\log_3 3}) = O(n^{1.59}) \checkmark \text{较大的改进}$$

1. $XY = ac \cdot 2^n + ((a-b)(d-c) + ac + bd) \cdot 2^{n/2} + bd$
2. $XY = ac \cdot 2^n + ((a+b)(c+d) - ac - bd) \cdot 2^{n/2} + bd$

细节问题：两个XY的复杂度都是 $O(n^{\log_3 3})$ ，但考虑到 $a+b, c+d$ 可能得到 $m+1$ 位的结果，使问题的规模变大，故不选择第2种方案。



10.3 例2-大整数的乘法

- ▶ 请设计一个有效的算法，可以进行两个n位二进制大整数的乘法运算
 - 小学的方法： $O(n^2)$ ✖效率太低
 - 分治法： $O(n^{1.59})$ ✔较大的改进
 - 更快的方法??
- ✔ 如果将大整数分成更多段，用更复杂的方式把它们组合起来，将有可能得到更优的算法。
- ✔ 这个思想导致了**快速傅利叶变换**(Fast Fourier Transform)的产生。该方法也可以看作是一个复杂的分治算法。



10.3 例3-矩阵乘法

► 传统方法：复杂度 $O(n^3)$

A和B的乘积矩阵C中的元素 $C[i,j]$ 定义为：
$$C[i][j] = \sum_{k=1}^n A[i][k]B[k][j]$$

若依此定义来计算A和B的乘积矩阵C，则每计算C的一个元素 $C[i][j]$ ，需要做n次乘法和n-1次加法。因此，算出矩阵C的个元素所需的计算时间为 $O(n^3)$



10.3 例3-矩阵乘法

▶ 传统方法：复杂度 $O(n^3)$

▶ 分治法：

使用与上例类似的技术，将矩阵A，B和C中每一矩阵都分块成4个大小相等的子矩阵。由此可将方程 $C=AB$ 重写为：

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

由此可得

复杂度分析

$$T(n) = \begin{cases} O(1) & n = 2 \\ 8T(n/2) + O(n^2) & n > 2 \end{cases}$$

$T(n)=O(n^3)$ **✗没有改进☹**

$$C_{22} = A_{21}B_{12} + A_{22}B_{22}$$



10.3 例3-矩阵乘法

▶ 传统方法：复杂度 $O(n^3)$

▶ 分治法：
$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

为了降低时间复杂度，必须减少乘法的次数。

$$M_1 = A_{11}(B_{12} - B_{22})$$

$$M_2 = (A_{11} + A_{12})B_{22}$$

$$M_3 = (A_{21} + A_{22})B_{11}$$

$$C_{11} = M_5 + M_4 - M_2 + M_6$$

M_4
 M_5
 M_6

复杂度分析

$$T(n) = \begin{cases} O(1) & n = 2 \\ 7T(n/2) + O(n^2) & n > 2 \end{cases}$$

M_7

$$T(n) = O(n^{\log_2 7}) = O(n^{2.81}) \quad \checkmark \text{较大的改进} \odot$$



10.3 例3-矩阵乘法

- ▶ 传统方法：复杂度 $O(n^3)$
- ▶ 分治法： $O(n^{2.81})$
- ▶ 更快的方法??

➤ Hopcroft和Kerr已经证明(1971)，计算2个 2×2 矩阵的乘积，7次乘法是必要的。因此，要想进一步改进矩阵乘法的时间复杂性，就不能再基于计算 2×2 矩阵的7次乘法这样的方法了。或许应当研究 3×3 或 5×5 矩阵的更好算法。

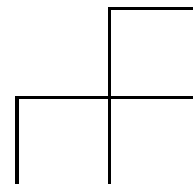
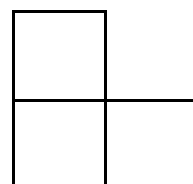
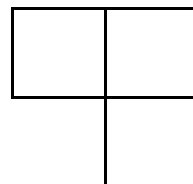
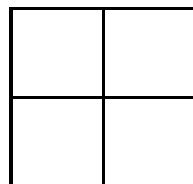
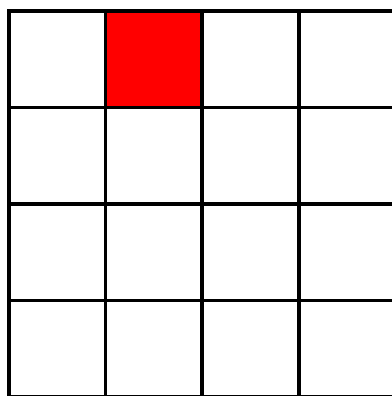
➤ 在Strassen之后又有许多算法改进了矩阵乘法的计算时间复杂性。目前最好的计算时间上界是 $O(n^{2.376})$

➤ 是否能找到 $O(n^2)$ 的算法??? 目前为止还没有结果。



10.3 例4-棋盘覆盖

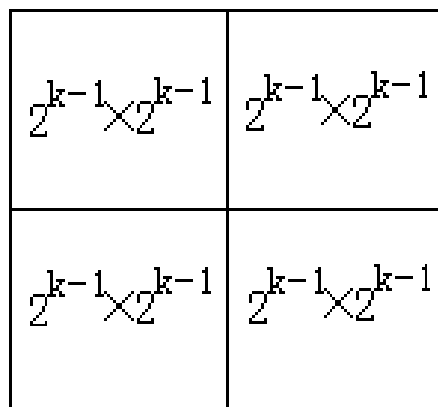
- 在一个由 $2^k \times 2^k$ 个方格组成的棋盘中，恰有一个方格与其它方格不同，称该方格为一特殊方格，且称该棋盘为一特殊棋盘。在棋盘覆盖问题中，要用图示的4种不同形态的L型骨牌覆盖给定的特殊棋盘上除特殊方格以外的所有方格，且任何2个L型骨牌不得重叠覆盖。



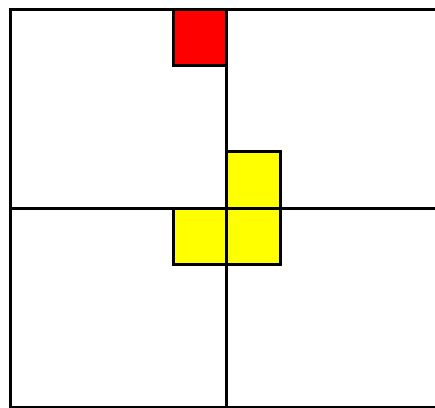


10.3 例4-棋盘覆盖

- ▶ (1) 当 $k > 0$ 时, 将 $2^k \times 2^k$ 棋盘分割为4个 $2^{k-1} \times 2^{k-1}$ 子棋盘(a)所示。
- ▶ (2) 特殊方格必位于4个较小子棋盘之一中, 其余3个子棋盘无特殊方格。为了将这3个无特殊方格的子棋盘转化为特殊棋盘, 可以用一个L型骨牌覆盖这3个较小棋盘的会合处, 如(b)所示, 从而将原问题转化为4个较小规模的棋盘覆盖问题。



(a)

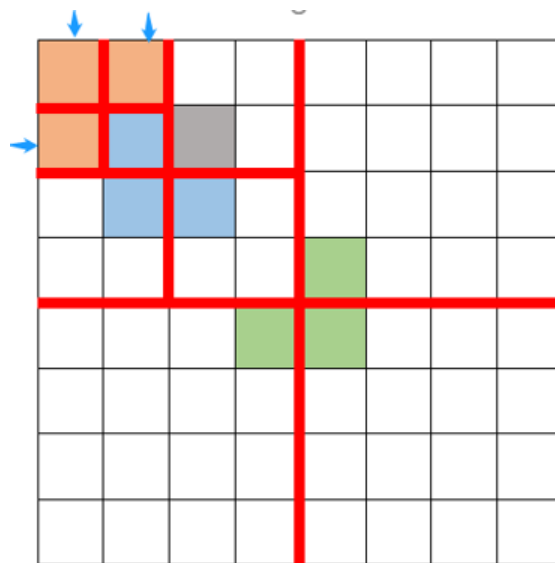
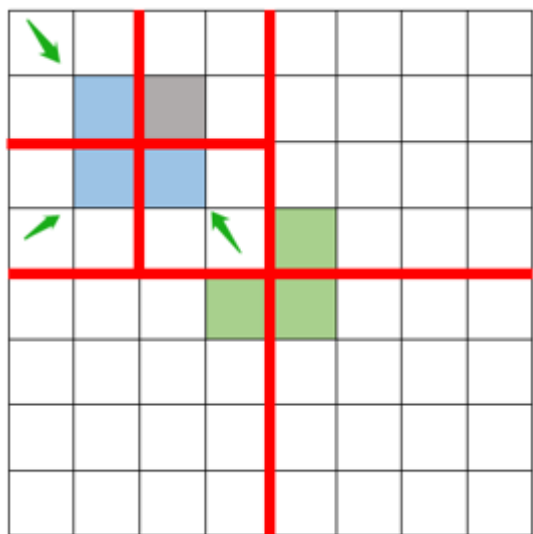


(b)



10.3 例4-棋盘覆盖

- ▶ (3) 取划分后的四个格子其中一个进一步划分，与(2)的原理一致。



- ▶ 递归地进行以上步骤，最终划分后的格子长度为1，已无法再进行4等分，则截止；
- ▶ 其余的3个4x4部分也依照此法依次划分



10.3 例4-棋盘覆盖

➤ tr/tc: 左上角方格行号/列号, dr/dc: 特殊方格行号/列号, size: 2^k

```
void chessBoard(int tr, int tc, int dr, int  
dc, int size)
```

```
{  
    if (size == 1) return;  
    int t = tile++, // L型骨牌号  
        s = size/2; // 分割棋盘  
    // 覆盖左上角子棋盘  
    if (dr < tr + s && dc < tc + s)  
        // 特殊方格在此棋盘中  
        chessBoard(tr, tc, dr, dc, s);
```

```
    else { // 此棋盘中无特殊方格  
        // 用 t 号L型骨牌覆盖右下角  
        board[tr + s - 1][tc + s - 1] = t;  
        // 覆盖其余方格  
        chessBoard(tr, tc, tr+s-1, tc+s-1,  
s);}  
    // 覆盖右上角子棋盘  
    if (dr < tr + s && dc >= tc + s)  
        // 特殊方格在此棋盘中  
        chessBoard(tr, tc+s, dr, dc, s);  
    else { // 此棋盘中无特殊方格  
        // 用 t 号L型骨牌覆盖左下角
```



10.3 例4-棋盘覆盖

```

board[tr + s - 1][tc + s] = t;           // 覆盖右下角子棋盘
// 覆盖其余
chessBoard
// 覆盖左下
if (dr >= tr +
    // 特殊方格
    chessBoard
else { // 用 t 号L型骨牌覆盖右上角           chessBoard(tr+s, tc+s, tr+s, tc+s, s);}
    board[tr + s][tc + s - 1] = t;           }
    // 覆盖其余方格
    chessBoard(tr+s, tc, tr+s, tc+s-1, s);}

```

复杂度分析

$$T(k) = \begin{cases} O(1) & k = 0 \\ 4T(k-1) + O(1) & k > 0 \end{cases}$$

$$T(k) = O(4^k)$$



10.3 例5：合并排序

基本思想：将待排序元素分成大小大致相同的2个子集合，分别对2个子集合进行排序，最终将排好序的子集合合并成为所要求的排好序的集合。

```
public static void mergeSort(Comparable a[], int left, int right)
{
    if (left < right) { // 至少有2个元素
        int i = (left + right) / 2; // 取中点
        mergeSort(a, left, i);
        mergeSort(a, i + 1, right);
    }
}
```

复杂度分析

$$T(n) = \begin{cases} O(1) & n \leq 1 \\ 2T(n/2) + O(n) & n > 1 \end{cases}$$

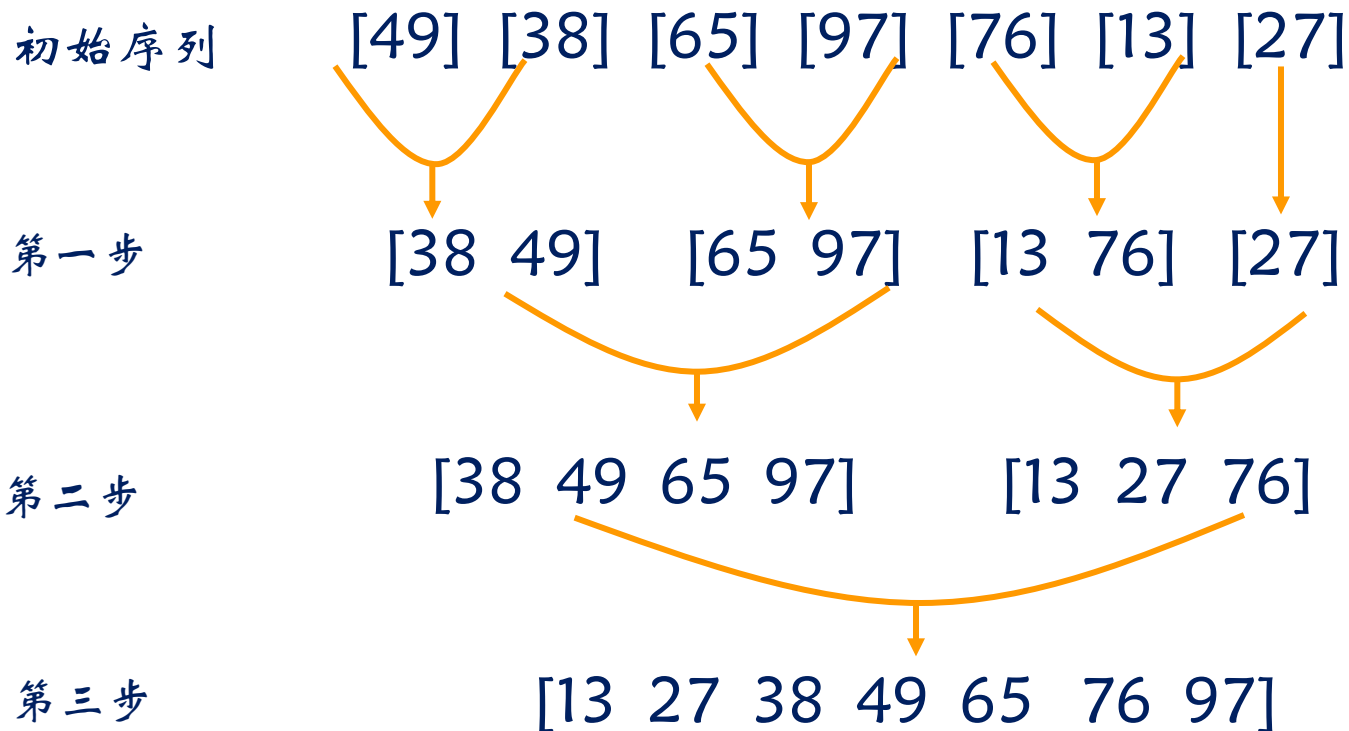
$$T(n) = O(n \log n)$$

非序



10.3 例5：合并排序

算法mergeSort的递归过程可以消去，即二路归并排序。





10.3 例5：合并排序

📖 最坏时间复杂度： $O(n \log n)$

📖 平均时间复杂度： $O(n \log n)$

📖 辅助空间： $O(n)$

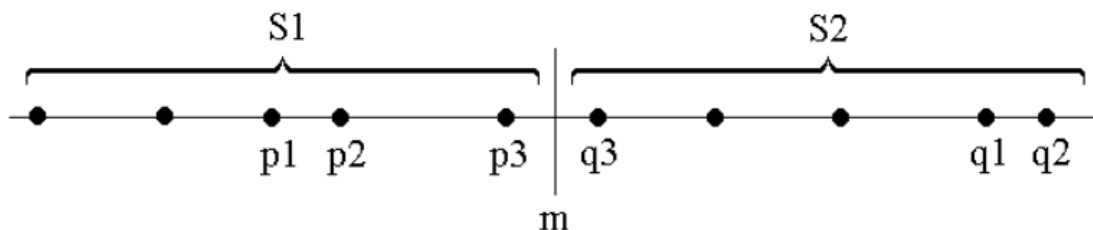
📖 稳定性：稳定



10.3 例6-最近点对问题

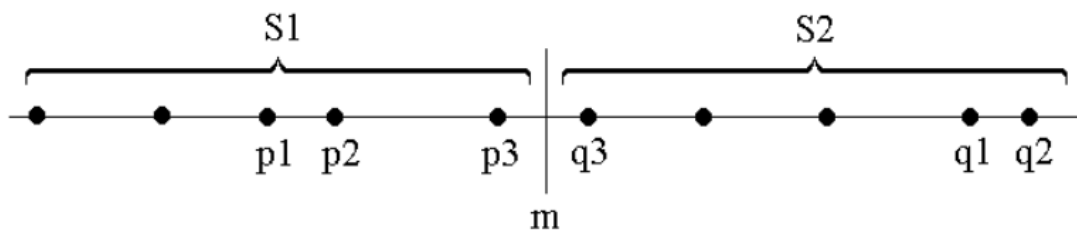
问题： 给定平面上 n 个点集 S ，找其中一对点，使得其在 n 个点组成的所有点对中，该点对间的**距离最小**

- 为了使问题易于理解和分析，**先来考虑一维的情形**。此时， S 中的 n 个点退化为 x 轴上的 n 个实数 $x_1, x_2, \dots, x_n$ ，最接近点对即为这 n 个实数中相差最小的2个实数
- 假设我们用 x 轴上某个点 m 将 S 划分为2个子集 S_1 和 S_2 ，基于平衡子问题的思想，用 S 中各点坐标的中位数来作分割点
- 递归地在 S_1 和 S_2 上找出其最接近点对 $\{p_1, p_2\}$ 和 $\{q_1, q_2\}$ ，并设 $d = \min\{|p_1 - p_2|, |q_1 - q_2|\}$ ， S 中的最接近点对或者是 $\{p_1, p_2\}$ ，或者是 $\{q_1, q_2\}$ ，或者是某个 $\{p_3, q_3\}$ ，其中 $p_3 \in S_1$ 且 $q_3 \in S_2$
- 能否在线性时间内找到 p_3, q_3 ？**





10.3 例6-最近点对问题



• 能否在线性时间内找到 p_3, q_3 ?

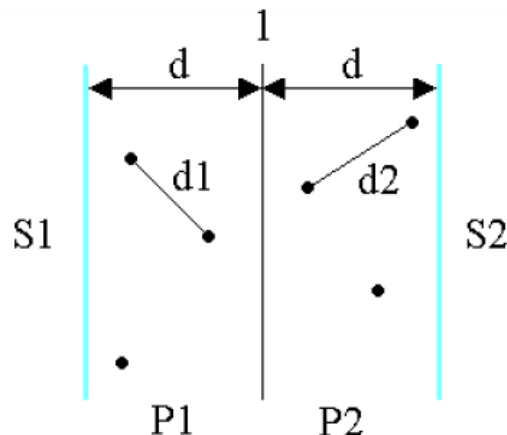
- 如果 S 的最接近点对是 $\{p_3, q_3\}$, 即 $|p_3 - q_3| < d$, 则 p_3 和 q_3 两者与 m 的距离不超过 d , 即 $p_3 \in (m-d, m]$, $q_3 \in (m, m+d]$
- 由于在 S_1 中, 每个长度为 d 的半闭区间至多包含一个点 (否则必有两点距离小于 d), 并且 m 是 S_1 和 S_2 的分割点, 因此 $(m-d, m]$ 中至多包含 S 中的一个点。由图可以看出, 如果 $(m-d, m]$ 中有 S 中的点, 则此点就是 S_1 中最大点。对 $(m, m+d]$ 区间同理。
- 因此, 我们用线性时间就能找到区间 $(m-d, m]$ 和 $(m, m+d]$ 中所有点, 即 p_3 和 q_3 。从而我们用线性时间就可以将 S_1 的解和 S_2 的解合并成为 S 的解



10.3 例6-最近点对问题

➤ 进一步，考虑二维的情形

- 选取一垂直线 $l: x=m$ 来作为分割直线，其中 m 为 S 中各点 x 坐标的中位数，由此将 S 分割为 S_1 和 S_2
- 递归地在 S_1 和 S_2 上找出其最小距离 d_1 和 d_2 ，并设 $d = \min\{d_1, d_2\}$ ， S 中的最接近点对或者是 d ，或者是某个 $\{p, q\}$ ，其中 $p \in S_1$ 且 $q \in S_2$
- 能否在线性时间内找到 p, q ? p 和 q 距离直线 l 的距离均小于 d
- 用 P_1 和 p_2 分别表示直线 l 左侧和右侧的两个垂直长条区域。极端情况下要检查 $n^2/4$ 点对。

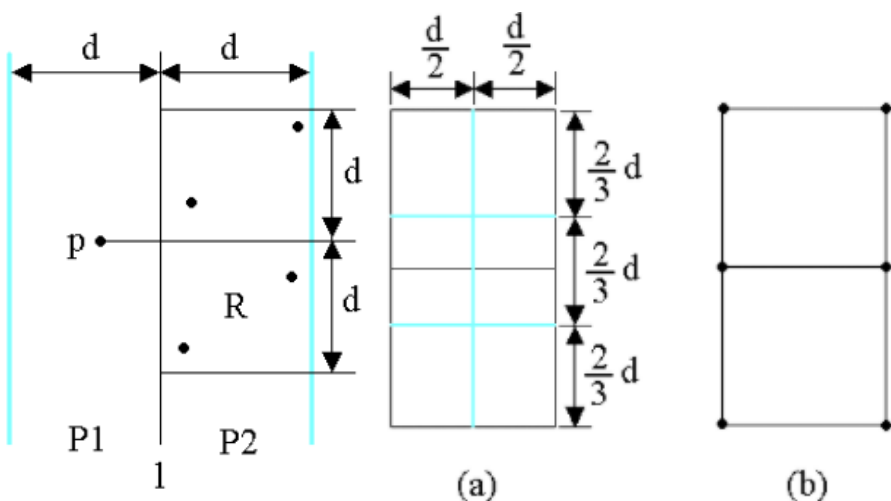




10.3 例6-最近点对问题

• 能否在线性时间内找到 p, q ?

- 考虑 P_1 中任意一点 p ，它若与 P_2 中的点 q 构成最接近点对的候选者，则必有 $\text{distance}(p, q) < d$ ，满足这个条件的 P_2 中的点一定落在一个 $d \times 2d$ 的矩形 R 中
- 由 d 的意义可知， P_2 中任何2个 S 中的点的距离都不小于 d ，由此可以推出矩形 R 中最多只有6个 S 中的点
- 因此，在分治法的合并步骤中最多只需要检查 $6 \times n/2 = 3n$ 个候选者



反证：将矩形 R 的长为 $2d$ 的边3等分，将它的长为 d 的边2等分，由此导出6个 $(d/2) \times (2d/3)$ 的矩形。若矩形 R 中有多于6个 S 中的点，则由鸽舍原理易知至少有一个 $(d/2) \times (2d/3)$ 的小矩形中有2个以上 S 中的点。设 u, v 是位于同一小矩形中的2个点，则

$$(x(u) - x(v))^2 + (y(u) - y(v))^2 < \left(\frac{d}{2}\right)^2 + \left(\frac{2d}{3}\right)^2 = \frac{25}{36}d^2$$

故 $\text{distance}(u, v) < d$



10.3 例6-最近点对问题

- 为了确切地知道要检查哪6个点，可以将 p 和 P_2 中所有 S_2 的点投影到垂直线 l 上
 - 由于能与 p 点一起构成最接近点对候选者的 S_2 中点一定在矩形 R 中，所以它们在直线 l 上的投影点距 p 在 l 上投影点的距离小于 d
 - 由上面的分析可知，这种投影点最多只有6个
- 因此，若将 P_1 和 P_2 中所有 S 中点按其 y 坐标排好序，则对 P_1 中所有点，对排好序的点列作一次扫描，就可以找出所有最接近点对的候选
 - 对 P_1 中每一点最多只要检查 P_2 中排好序的相继6个点



10.3 例6-最近点对问题

```
double cpair2(S)
```

```
{
```

```
    n=|S|;
```

```
    if (n < 2) return ∞;
```

1、m=S中各点x间坐标的中位数;

构造S1和S2;

```
//S1={p∈S|x(p)≤m},
```

```
//S2={p∈S|x(p)>m},
```

2、d1=cpair(S1)

d2=cpair(S2)

3、dm=min(d1,d2)

4、设P1是S1中距垂直分割线l的距离在dm之内的所有点组成的集合;

P2是S2中距分割线l的距离在dm之内所有点组成的集合;

将P1和P2中点依其y坐标值排序;

并设X和Y是相应的已排好序的点列;

5、扫描X以及对X中每个点检查Y中与其距离在dm之内的所有点(最多6个)完成合并;

复杂度分析

$$T(n) = \begin{cases} O(1) & n < 4 \\ 2T(n/2) + O(n) & n \geq 4 \end{cases}$$

$$T(n) = O(n \log n)$$

return d;



10.3 例7-循环赛日程表

- ▶ 设计一个满足以下要求的比赛日程表：
 - ▶ (1) 每个选手必须与其他 $n-1$ 个选手各赛一次；
 - ▶ (2) 每个选手一天只能赛一次；
 - ▶ (3) 循环赛一共进行 $n-1$ 天。

按分治策略，将所有的选手分为两半， n 个选手的比赛日程表就可以通过为 $n/2$ 个选手设计的比赛日程表来决定。递归地对选手进行分割，直到只剩下2个选手时，比赛日程表的制定就变得很简单。这时只要让这2个选手进行比赛就可以了。

1	2	3	4	5	6	7	8
2	1	4	3	6	5	8	7
3	4	1	2	7	8	5	6
4	3	2	1	8	7	6	5
5	6	7	8	1	2	3	4
6	5	8	7	2	1	4	3
7	8	5	6	3	4	1	2
8	7	6	5	4	3	2	1



10.3 例7-循环赛日程表

► 递归实现

1	2	3	4	5	6	7	8
2	1	4	3	6	5	8	7
3	4	1	2	7	8	5	6
4	3	2	1	8	7	6	5
5	6	7	8	1	2	3	4
6	5	8	7	2	1	4	3
7	8	5	6	3	4	1	2
8	7	6	5	4	3	2	1

► 1-8排赛分成:

- 1-4排赛和5-8排赛

- 右上角紫色矩阵拷贝左下角矩阵, 右下角褐色矩阵拷贝左上角矩阵

► 1-4排赛分成:

- 1-2排赛和3-4排赛

- 右上角黄色矩阵拷贝左下角矩阵, 右下角橙色矩阵拷贝左上角矩阵

► 1-2排赛:

- 对应赋值.



10.3 例7-循环赛日程表

► 递归实现

```
const int maxn = 10000;
int a[maxn][maxn]
inline void dfs(int n, int k){ //n:队员数, k:起始队员(0开始编号)
    if (n == 2){ //若只剩下2名队员, 直接赋值2*2矩阵
        a[k][0] = k+1; a[k][1] = k+2;
        a[k+1][0] = k+2; a[k+1][1] = k+1;
    }
    else{
        dfs(n/2, k); //从k~k+n/2-1的队员编赛
        dfs(n/2, k+n/2); //从k+n/2~n-1的队员编赛
        for (int i=k; i<k+n/2; i++){ //拷贝到右上角
            for (int j=n/2; j<n; j++) a[i][j] = a[i+n/2][j-n/2];
        }
        for (int i=k+n/2; i<k+n; i++){ //拷贝到右下角
            for (int j=n/2; j<n; j++) a[i][j] = a[i-n/2][j-n/2];
        }
    }
}
```

```
int main()void dfs(int n, int k)
{
    int n;
    while (scanf("%d",&n)!=EOF){
        dfs(n,0);
        for(int i=0; i<n;i++){
            for (int j=0; j<n;j++){
                printf("%d", a[i][j]);
                printf("\n");
            }
        }
    }
}
```




10.3 例7-循环赛日程表

▶ 非递归算法实现

1	2	3	4	5	6	7	8
2	1	4	3	6	5	8	7
3	4	1	2	7	8	5	6
4	3	2	1	8	7	6	5
5	6	7	8	1	2	3	4
6	5	8	7	2	1	4	3
7	8	5	6	3	4	1	2
8	7	6	5	4	3	2	1

右边上下矩阵

- ▶ n 个 $1*1$ 矩阵拷贝
- ▶ $n/2$ 个 $2*2$ 矩阵拷贝
- ▶ $n/4$ 个 $4*4$ 矩阵拷贝

$n=8$



Next: 贪心算法

